

# 19 Dimensional Scalar WordNet Context Propagation (v2.0)

Cody Michael Allison (Michael Rendier)

## Contents

|                                                                                     |          |
|-------------------------------------------------------------------------------------|----------|
| <b>19 Dimensional Scalar WordNet Context Propagation (v2.0)</b>                     | <b>1</b> |
| Abstract . . . . .                                                                  | 2        |
| 1. The problem . . . . .                                                            | 2        |
| 2. Notation, disambiguation, and install . . . . .                                  | 3        |
| 3. “The primes are the words” . . . . .                                             | 4        |
| 4. Why primes at all — how we came to 313 . . . . .                                 | 8        |
| 5. Spelling → prime — order, and the anagram collision . . . . .                    | 9        |
| 6. monad3_c.bin — built to carry the 19D WordNet relationships . . . . .            | 11       |
| 7. Reading monad3_c.bin’s WordNet relationships . . . . .                           | 13       |
| 8. Cost, measured — and why there was no cost function to measure against . . . . . | 15       |
| 9. The Pencil — windspeed, and what it takes to run the box kite today . . . . .    | 18       |
| 10. Conclusion — what the scalar actually does, proven and open . . . . .           | 20       |
| 11. Direction of engineering research — in development . . . . .                    | 22       |
| 12. Provenance and attribution . . . . .                                            | 23       |
| 13. Code appendix . . . . .                                                         | 26       |

## 19 Dimensional Scalar WordNet Context Propagation (v2.0)

**Cody Michael Allison**<sup>1</sup> (Michael Rendier)

<sup>1</sup> Independent researcher. Correspondence: [the.wandering.god@gmail.com](mailto:the.wandering.god@gmail.com) · GitHub: [github.com/michaelrendier](https://github.com/michaelrendier) · ORCID: [0009-0007-7239-6760](https://orcid.org/0009-0007-7239-6760).

**Fourth Age Paper.** One Paper. One Structure. One Engine. One Wiki.

**v2.0, 2026-09-20** — extends v1.0 with the windspeed reconstruction proof of concept (§9), the box-kite build and read chain from WordNet through the live C monad (§6, §13), and a full conclusion / future-work split (§10, §11). Structured as a converted notebook, in the spirit of *The Annotated Transformer* (Harvard NLP) — prose interleaved with the real code that produced each result, in the order it was found.

## Abstract

Large language models represent a word’s context as a single dense vector, learned end-to-end and opaque to inspection — the well-documented difficulty of recovering interpretable, per-token structure from a transformer’s internal representations (superposition, polysemantic features) is one symptom of a more basic absence: there is no explicit, multidimensional, per-word context representation that can be read, audited, or composed independently of the weights that produced it. With the help of Claude-Code as a coding, research and calculation tool; I present a fully implemented, deterministic alternative. Each word in the Open English WordNet (146,743 synsets) is mapped to a single prime number that jointly and losslessly encodes its exact spelling and its full 19-dimensional WordNet relational signature — hypernymy, meronymy, and the sixteen other relation types WordNet defines — with no learned parameters and no training step. The encoding is recovered exactly from the prime: 146,743/146,743 synsets round-trip exact (100.000%) against the current build, with one deliberate, quantified lossy step affecting 1.1% of individual relation counts. The method was engineered independently, as a self-contained process tree built from first principles to solve the problem of hashing structured context into word representations; correspondences identified afterward to established mathematics — projective-plane combinatorics, Gödel positional encoding, Miller-Rabin primality — are cited throughout as such, not presented as the method’s starting point. The construction generalizes to a class of zero-divisor combinatorial structures (box kites, in sedenion algebra) that have so far only been described by fixing their internal positions directly; here, the internal structure is instead read from the deformation of the structure’s outer boundary under a single continuous parameter, reducing many interpretable dimensions to one real number per word. Components resting on established mathematics ship, tested against a live 347,119-word vocabulary; where the reduction remains an open construction, it is marked THEORETICAL, and where our own code already calculates a concrete result about that open piece, the label is refined to THEORETICAL:CALCULATED rather than allowed to imply more than is proven.

---

## 1. The problem

A transformer’s representation of a word’s context is a fixed-length array of floating-point numbers — a few thousand float32s per token, produced by a chain of matrix multiplications and set entirely by gradient descent during training. Nothing about that array is addressable the way an ordinary data structure is. There is no field you can name and read off independently — no `is_a`, no `part_of`, no domain you could log, assert against in a test, or diff between two runs. Every one of those thousands of numbers depends on every other one; changing what the vector “means” means retraining, not editing a field. It is a value with no schema.

This is not a complaint about accuracy — the vectors work, in the sense that models trained on them perform well. It is a complaint about **engineering interface**: nothing about a token embedding is inspectable, versionable, or composable the way a struct, a database row, or a hash is. You cannot grep it. You cannot write assert

`context.is_a("mammal")` and have it mean anything. You cannot cheaply diff two words' contexts and get back which of a fixed set of named relations differ.

WordNet already has exactly the schema that's missing — 19 named relation types per synset (hypernym, meronym, entailment, domain, and the rest), curated by hand, stable across builds. What doesn't exist is a way to carry that schema forward *as an address*: something as small and portable as a hash or an integer, computed with no training step, exactly recoverable back into the 19 named fields it came from. That gap — a compact, deterministic, fully-recoverable address for a word's WordNet relational record, plus its exact spelling, in one value — is what the rest of this paper builds, one piece of code at a time.

---

## 2. Notation, disambiguation, and install

### 2.1 Notation

Every component described in this paper carries one of five labels, applied consistently rather than asserted once in prose and forgotten:

- **ESTABLISHED** — mathematics or computer science that predates this project, cited to its source.
- **OURS** — code or a specific design choice built for this project, not published elsewhere.
- **FIRST STATED HERE** — a specific claim or construction, dated, not found stated this way anywhere else the author has checked.
- **THEORETICAL** — designed, or partially attempted, but not yet reduced to running, verified code.
- **THEORETICAL:CALCULATED** — a **THEORETICAL** component for which code has nonetheless computed a concrete, reportable result (a measurement, a boundary condition, a failed literal construction) without the component itself shipping. The distinction matters because a **THEORETICAL:CALCULATED** claim is falsifiable and reproducible today, even though the larger construction it belongs to is not.

### 2.2 Disambiguation — the code came first

Every piece of code in this paper was built directly against the problem in §1, without first consulting the literature it turns out to correspond to. Correspondences to established mathematics — projective-plane combinatorics, Gödel positional encoding, Miller-Rabin primality, sedenion zero-divisor structures — were noticed *after* the code already worked, by checking its output against the literature, not used to construct the code in the first place. Where a mathematical name appears below, it is a label applied after the fact for a reader who already knows that name; it played no role in how the code was written, and nothing in this paper requires knowing it to follow the code. The reverse is stated with the same care: the small number of pieces marked **FIRST STATED HERE** are exactly that — not found named this way anywhere else the author has checked, not claimed as more than that either.

## 2.3 Install

```
python3 -m venv .venv
source .venv/bin/activate
pip3 install -r requirements.txt
python3 -c "import nltk; nltk.download('wordnet')"
```

requirements.txt is in this directory and is short (nltk, jupyter) — it installs the same way on Linux, macOS, and Windows. The five notebooks under notebooks/ are the executable form of every result in this paper; each runs directly once the venv above is active and this repository’s sibling repos (VAPMIP, ValaQuenta) are checked out alongside it.

---

---

## 3. “The primes are the words”

Every mechanism in this paper eventually addresses a word by its *relational* structure — what it means in context, not how it’s spelled. That wasn’t the starting idea. The starting idea, in VAPMIP/monad.py since 2026-05-27, unaltered since, was simpler and more ambitious at once: that a prime number could stand for a **concept**, the same irreducible way it stands for a number — and that mapping a word onto one would be enough, on its own, to place related words near each other, across languages, because the concept was assumed to be prior to any particular spelling of it. §3.4 is honest about what running that idea actually produced: a field of collisions dense enough that the scripting around it was shelved — not because the idea was “wrong” in some final sense, its success or failure was never really the point, but because that density is what pointed toward relational *structure*, not a bare address, as the place to keep looking. This section is the mechanical piece underneath that whole arc: a deterministic, collision-mostly- avoiding map from a word’s spelling to an address, read from the code that has run, unmodified, for four months.

### Two real screenshots, not staged for this paper — the actual first runs:

**Figure 3a.** SpaceClaude/funny-ptolemy.png — the very first time this machinery was ever run: `ptolemy -l 'what is your name' → Segmentation fault (core dumped)`. Kept in the record on purpose, not cropped out: this is what “the first attempt at semantic hash” actually looked like, the honest starting point before any of the rest of this paper existed.

**Figure 3b.** SpaceClaude/TheExtractor.png — the first real question put to the running system, asked two ways. `ptolemy -h 'what is your name'` prints the per-word state table ( $z\#, \gamma, \sigma=0.5, E, \beta=7.552000$  — the same MONAD\_BETA\_SAT cited in §8, visible here from the very first run); `ptolemy -W 'what is your name'` then returns `holcussamestuffseemy`. This is where the name **Holcus** came from — a real word (a grass genus, “Yorkshire fog”) surfacing out of the very first multilingual semantic-prime-hash run, not chosen in advance.

### 3.1 The address, in the four lines it actually is

```
# VAPMIP/monad.py – unaltered since commit 204c75d, 2026-05-27
_PRIME_CAP = 1 << 16                                # 65536 – primes searched in [2, 65537]

def _horner_hash(w: str, base: int = 95, offset: int = 32) -> int:
    v = 0
    for ch in w:
        v = v * base + max(0, ord(ch) - offset)
    return abs(v)

def _word_zero_idx(w: str) -> int:
    v = _horner_hash(w)
    p = _next_prime(v)                                # smallest prime >= v mod _PRIME_CAP
    idx = _prime_pi_table[min(p, _PRIME_CAP + 1)]    # pi(p) -- count of primes <= p
    return max(1, idx)
```

Per [cs-paper-code-conventions](#) §1: this is OURS, short, and is the paper’s contribution at this step — it gets the real listing, not a  $\Sigma$  standing in for a loop. Three established pieces do the actual work, cited and not re-derived: a Horner polynomial evaluation (base 95, the size of printable ASCII) turns a word’s spelling into one integer; a sieve of Eratosthenes, run once at import, supplies both a fast `next_prime` and a precomputed  $\pi$  table (count of primes  $\leq k$ , for every  $k$  up to 65536); composing them sends a word to **which of up to 6,543 non-trivial Riemann zeros indexes its address** — the pencil selector every later section of this paper reads off of.

```
>>> for w in ["accretion", "pile", "toroidal", "condensate", "narrative", "dissertation"]
...     trace(w)
'accretion' H= 435952031571538408 next_prime= 7933 idx= 1002 gamma=1439.0154
'pile' H= 69256114 next_prime= 49043 idx= 5042 gamma=5548.5164
'toroidal' H= 5924746544723326 next_prime= 13513 idx= 1601 gamma=2117.7541
'condensate' H= 42756310149759033674 next_prime= 3761 idx= 523 gamma=853.14356
'narrative' H= 522068046232455314 next_prime= 1847 idx= 283 gamma=522.36692
'dissertation' H=391207708623858272995283 next_prime= 46919 idx= 4847 gamma=5365.31
```

(notebooks/01\_semantic\_prime\_hashing.ipynb, cell 3, run live against the unmodified function.) Determinism holds — the same word produces the same index on every call, checked against three repeated calls each for ten words, no exceptions.

### 3.2 The domain, measured on the live vocabulary

Run against all 347,119 words in VAPMIP/PtolC/monad3\_c.bin, at 109,403 words/second:

```
idx range: [1, 6543] distinct buckets used: 6543/6542
words per bucket: min 1 max 395 mean 53.1
most-populated buckets: [(3386, 395), (3645, 338), (4523, 310), ...]
```

Read that second number twice: 6543/6542. Not a typo — the code’s own count of

distinct buckets used *exceeds* the paper’s previously-stated bucket count. Chased rather than rounded away (notebooks/01...ipynb, cells 11–13):

```
>>> print("pi(65536) =", monad._prime_pi_table[65536],
...       " pi(65537) =", monad._prime_pi_table[65537])
pi(65536) = 6542   pi(65537) = 6543
```

`next_prime` is documented, and coded, to reach **65537** inclusive. 65537 is itself prime (it’s a Fermat prime,  $2^{16} + 1$ ) — so  $\pi(65537) = \pi(65536) + 1 = 6543$ , one past where the code’s own stated range said it could go. **Verdict: MATHS/METHOD, not CODE.** The function is internally consistent; it is two *stated* boundaries ( $p \in [2, 65537]$  and  $\text{idx} \in [1, 6542]$ ) disagreeing with each other. This is not a corner case nobody hits — 91 real words in the live vocabulary land in bucket 6543, most of them ordinary English (apocope, contemplating, discipline), a few tokenizer noise from the live corpus ('kay, 1387–1422, raw LaTeX fragments) worth naming honestly rather than pretending the vocabulary is cleaner than it is. The corrected range carried forward from here on:  $\text{idx} \in [1, 6543]$ , not 6542.

### 3.3 A live bug, reproduced, not fixed

`_horner_hash` clamps digits from *below* only ( $\max(0, \text{ord}(\text{ch}) - 32)$ ). An invisible Unicode character and a printable two-character sequence can therefore land on the same clamped digit:

```
>>> zwsp = "      # U+200B, ZERO WIDTH SPACE
>>> print(f"horner(U+200B) = {monad._horner_hash(zwsp)}"
...       f"   horner('v!') = {monad._horner_hash('v!')}")
horner(U+200B) = 8171   horner('v!') = 8171
>>> print(f"idx(U+200B) = {monad._word_zero_idx(zwsp)}"
...       f"   idx('v!') = {monad._word_zero_idx('v!')}")
idx(U+200B) = 1026   idx('v!') = 1026
```

VAPMIP/prime\_hash.py (2026-08-18) names what’s actually wrong here — a category error, not an off-by-one: whitespace and invisible control characters are **aperture** (they set which domain of words is being drawn from) and were never supposed to reach the letter polynomial as a digit at all. Its proposed fix, `split_tiers()`, strips anything outside printable ASCII before hashing, so an invisible character can never forge a collision. That fix has been designed and is not, as of this writing, wired into `monad.py` — the bug above is live in the function every later section of this paper still calls.

### 3.4 The historic foothold — the entrance to the freeway, not a verdict

The idea that a prime could stand for an **irreducible concept** — not per-language, but underneath every language — predates all of the above. The clearest statement of it is Ainulindale/wiki/16\_semantic\_word\_engine.md: tree/arbre/木/قرجش/Baum, water/eau/aqua/wasser, all claimed to land on the same invariant, “the concept is the prime, the language is the coordinate choice.” What’s recorded below is not a verdict on that idea — whether it succeeds or fails was never the point. It’s engineering

history: this mechanism, run for real, produced a field of collisions dense enough that the scripting around it was shelved for months, and picked back up only after a different object — the box kite, §10’s hyper-relationship snapshot — made it clear *why* the collisions were so dense. That pivot, not a pass/fail on wiki-16’s claim, is what this section is actually recording.

Run blind, today, against the unmodified mechanism above (repo\_appendix/semantic\_prime\_hash/05-27\_word\_zero\_idx.py):

```
=== tree ===
  en 'tree'      idx=2273
  es 'árbol'     idx=111
  zh '树'        idx=2919
=== water ===
  en 'water'     idx=2009
  fr 'eau'       idx=4122
  ja '水'        idx=3018
=== water/eau/aqua/wasser (wiki-16's own cited example) ===
  'water'       idx=2009
  'eau'         idx=4122
  'aqua'        idx=4310
  'wasser'      idx=1204
```

(re-run live, this session, against the unmodified function.)

None of the tested groups share an idx — including wiki-16’s own cited water/eau/aqua/wasser example. `_word_zero_idx` places every word *somewhere* (mechanically true by construction — it’s a total function), but a base-95 Horner hash over UTF-8 code points has no reason to be language-invariant, and isn’t. That’s not the interesting part, though — the interesting part, engineering-wise, is the *density* of collision this style of address produces once you’re not just checking a handful of hand-picked pairs but running it across a real vocabulary: enough collisions, close enough together, that continuing to script against a bare address stopped being productive.

The same signal shows up a second time, independently, three months later, in `wordnet_boxkite.py`’s own in-source history comment: an earlier version of that file’s collision test checked whether a synset and its hypernym produced *small prime gaps* — which, read back later, was quietly testing semantic relatedness rather than what was actually wanted. Cody’s own note, kept in the file rather than edited away: “*twin primes were to illustrate the idea of contextual neighborhood... not semantic neighborhood.*” What the retest actually turned up: synsets with an **exactly identical 19-relation shape** collide on address every time, regardless of what they mean — `hilbert.n.01` (a mathematician), `irrawaddy.n.01` (a river), and `new_york.n.03` (a city) share one address purely because all three have the same named-instance relational shape.

Both times, the collision density is what mattered, not a verdict on the mechanism that produced it — it’s what shelved the scripting the first time, and it’s what pointed at *relational structure* as the place worth building on the second. Everything from §9

onward (the pencil, the box kite, the WordNet relational vector) exists because that density was followed rather than argued with. That’s the sense in which this section is a foothold, not a result: it’s the on-ramp, not the destination — “the entrance to the freeway” of getting the maths to speak English.

---

#### 4. Why primes at all — how we came to 313

The question, asked plainly, before any of the rest of this paper existed: **what is the last prime number to see any new factors come down in the sieve?** — equivalently, the highest prime factor ever required to build any composite in the domain at all. Not offhand maths; a dataset of two conjugate domains over the composites (The Two Trees engine — primes, composites), built and checked directly:

```
# GenerationalLineage/engine/lineage.py::un_sieve, run for N=10^5
import lineage
r = lineage.un_sieve(100_000)
```

The standard optimised sieve only needs to strike multiples of  $p$  starting at  $p^2$  — every smaller multiple of  $p$  was already struck by a smaller prime factor. So  $p$  contributes a **new** strike only while  $p^2 \leq N$ . Direct code, not looked up:

```
>>> primes = sieve_primes(100_000)
>>> max(p for p in primes if p * p <= 100_000)
313
```

313 is the answer: the last prime with anything new to strike at  $N = 10^5$ . What that boundary actually splits the primes into, checked directly rather than reasoned about in prose (`_spf_gpf_tables(100_000)`, every composite  $\leq 10^5$ , no sampling):

```
>>> spf, gpf, primes = _spf_gpf_tables(100_000)
>>> comps = [n for n in range(4, 100_001) if spf[n] != n]
>>> max(spf[n] for n in comps)
313
>>> len({spf[n] for n in comps})           # every one of the 65 gets used
65
>>> never_spf = {p for p in {gpf[n] for n in comps} if p not in {spf[n] for n in comps}}
>>> len(never_spf), min(never_spf), max(never_spf), all(p > 313 for p in never_spf)
(5068, 317, 49999, True)
```

**Composite primes** — Telperion’s,  $\leq 313$ , 65 of them — are the complete, closed alphabet of every composite’s smallest-factor identity: no composite in the whole domain ever has an spf outside this set, and every one of the 65 gets used. **Factoral primes** — Laurelin’s, 317 through 49999, 5,068 of them — are exactly the primes that *never* once serve as an spf for anything: Telperion is permanently blind to them, zero exceptions, not approximately; they only ever surface as a gpf, which is to say only Laurelin ever accounts for them at all. The split isn’t a rule of thumb, it’s exact:  $\max(\text{spf}) == 313$ , always, for every composite  $\leq N$ , because  $\text{spf}(n) \leq \sqrt{n} \leq \sqrt{N}$  is arithmetic, not policy.

That single number is what turned an open question — are the primes *words*, or are they *letters* — into a decision. **The first 65 primes are letters.** With a bounded, principled pool to draw from (not an arbitrary round number, the sieve’s own extinction boundary), spelling can be hashed into primes in an orderly way, mostly collision-independent rather than colliding by accident — which is exactly where §5 picks up.

Reported for completeness, not developed further: the mirror question (when does a composite’s *largest* prime factor finish switching new composites on) doesn’t resolve until 49999 — a genuinely different, much larger number, left here as one honest line for whichever number theorist or physicist wants to run with it, not part of this paper’s argument.

---

## 5. Spelling → prime — order, and the anagram collision

§4 gave a closed, complete pool of “letter” primes to spend. The obvious way to spend it — one prime per letter, multiply the word’s letters together — is the first thing tried, and it’s broken in a specific, demonstrable way: **multiplication commutes**. A product of primes records *which* letters appear and *how many times*, by unique factorisation, and nothing else. It cannot, structurally, carry the *order* they appeared in. 'dog' != 'god'; a hash meant to reconstruct spelling exactly can’t lose that.

### 5.1 The naive scheme, and where it breaks

One prime per letter, alphabetically, product of the word:

```
_primes = sieve(200)
LETTER_PRIME = {chr(ord('a') + i): _primes[i] for i in range(26)}
# a=2, b=3, c=5, d=7, e=11, ... z=101

def naive_phonetic_hash(word: str) -> int:
    prod = 1
    for ch in word.lower():
        if ch.isalpha():
            prod *= LETTER_PRIME[ch]
    return prod
```

Run against real words (notebooks/03...ipynb, cell 2):

```
'cat' -> 710
'act' -> 710
'tac' -> 710
```

cat, act, and tac — three orderings of the same three letters — all hash to exactly 710. That’s not a near-miss, it’s every permutation landing on the same integer, every time, by construction:

```
naive_phonetic_hash('cat') == 710
naive_phonetic_hash('act') == 710
equal: True
```

```
naive_phonetic_hash('listen') == 1914801911
naive_phonetic_hash('silent') == 1914801911
equal: True
```

Every anagram pair in English collides. A prime-per-letter product is an excellent hash of the *multiset* of a word's letters and a total failure as a hash of the *word*.

## 5.2 The fix — position picks the base, the letter is the exponent

VAPMIP/wordnet\_boxkite.py::spelling\_code inverts which part of the construction carries which information. Position, not letter identity, selects the prime; the letter becomes the exponent on it:

```
LETTER_CAP = 71 # NOTE: 20-prime tier, not §4's 65
LETTER_PRIMES = [p for p in _P if p <= LETTER_CAP] # [2,3,5,7,...,71], 20 of them

def spelling_code(word: str) -> int:
    code = 1
    for i, ch in enumerate(w for w in word.lower() if w.isalpha()):
        p = LETTER_PRIMES[i % len(LETTER_PRIMES)] # position -> base, cycling every
        exp = ord(ch) - ord('a') + 1 # letter -> exponent, a=1..z=26
        code *= p ** exp
    return code
```

The 20 primes are one **per position**, not one per letter — the alphabet is not truncated. All 26 letters are fully carried, at every position, by the exponent (`ord(ch)-ord('a')+1`,  $a=1..z=26$ ); a lone letter's `spelling_code` is just that base's exponent, confirmed for all 26:  $a \rightarrow 2$ ,  $b \rightarrow 4$ ,  $c \rightarrow 8$ ,  $d \rightarrow 16$ , ...  $z \rightarrow 67108864$ . What the 20 actually bounds is **word length**: past the 20th letter, a position reuses an earlier position's base prime, and the two positions' exponents land on the same base and simply **add**. Built a real one, not a hypothetical — two distinct 21-letter words, position 0 and position 20 sharing base prime 2,  $1+3 == 2+2$ :

```
>>> spelling_code('a' + 'x'*19 + 'c') # pos 0='a'(1), pos 20='c'(3) -> 1+3=4 on base
>>> spelling_code('b' + 'x'*19 + 'b') # pos 0='b'(2), pos 20='b'(2) -> 2+2=4 on base
True # genuine collision — not because a letter is missing, because a position repeats
```

Position *i*'s prime is fixed regardless of which letter sits there; swapping two letters now swaps which *exponent* lands on which *base* — no longer a symmetric product, order survives (for words that don't reach the wraparound above):

```
'cat'      spelling_code = 2288818359375000
'act'      spelling_code = 5149841308593750
'listen'   spelling_code = 77805891137496681806187293144874121384823203125000000000000
'silent'   spelling_code = 305601935552050069509298342748495166804755637888000000000000

cat == act now?    False
listen == silent?  False
```

Exact, order-sensitive spelling as a prime, recoverable — position *i*'s prime is known

in advance, so factoring the code back at each of those 20 recurring bases recovers each letter's exponent, in order:

```
'cat'           -> decoded 'cat'           match: True
'act'           -> decoded 'act'           match: True
'windspeed'     -> decoded 'windspeed'     match: True
'reconstructible' -> decoded 'reconstructible' match: True
```

### 5.3 Measured, not asserted — live 30,000-word sample

```
exact word recovery: 27,375/28,728 (95.290%)
words <=20 alpha chars: 28,296 exact 27,375 (96.745%)
words >20 alpha chars: 432 (position cycle wraps at 20 -> exponents
                        add on the reused prime -> lossy, by
                        construction)
```

The failure mode is exact and predicted, not mysterious: the tier is 20 primes, cycling. A word over 20 letters reuses a base prime at two different positions, and since a product of prime powers can't distinguish  $p^a \cdot p^b$  from  $p^{(a+b)}$ , those two positions' exponents just add — recoverable as a sum, not as two separate letters. Lossy exactly where the construction says it will be, and nowhere else: **96.745%** exact on the  $\leq 20$ -letter words the scheme is actually built for, **0%** guaranteed-exact past that by the same arithmetic that gives the 96.745%.

### One honest note carried forward

§4 established 65 primes ( $\leq 313$ ) as the complete letter pool. What actually ships here uses a smaller, 20-prime cycling tier ( $\leq 71$ ) — the 20-prime tier is what the position-cycling needs (enough bases that most real words never wrap), not the full 65. Worth being upfront about in the paper rather than letting a reader notice the numbers don't match: is the 65-prime pool meant to become the tier size eventually (fewer wraps, longer words safe), or is 20 the settled design and 65 is a separate fact about the domain, not a promise about this tier's size?

---

## 6. monad3\_c.bin — built to carry the 19D WordNet relationships

Pigeon-holed deliberately: this section is how monad3\_c.bin came to carry the WordNet box-kite table, nothing wider. Explicitly **out of scope here** — bootstrapping and bulk corpus ingest (ingest.c, build\_monad\_bin.py merge), the filesystem-ingestion experiments and the .bin poisoning that followed from pointing a bulk ingest at a real computer's own file structure, and the vocabulary-update law's formulas (what the Monad's live "learn" step actually computes). Named here so the boundary is visible to a reader, not silently narrowed.

## 6.1 The WordNet box-kite table, built in C

VAPMIP/PtolC/boxkite\_bin.h defines the shared record — one BoxKiteEntry per WordNet synset, 19 compressed relation exponents plus a depth weight, the same schema wordnet\_boxkite.py uses on the Python side (documented as “two different serialisations of one schema, not two different schemas”):

```
typedef struct {
    char    word[32];
    uint8_t pos;                /* 1=NOUN 2=VERB 3=ADJ 4=ADV */
    uint32_t synset_offset;     /* WordNet's own offset -- the stable id */
    int16_t  vector[19];        /* compress_count()-ed relation exponents */
    float    depth_weight;      /* default 1.0 */
} BoxKiteEntry;
```

VAPMIP/PtolC/dump\_boxkite\_bin.c builds c\_monad\_wordnet.bin from that struct — reading WordNet’s own index.noun/verb/adj/adv files directly (not a library iterator; getindex() is fuzzy search, not enumeration), calling the real C WordNet library (index\_lookup, read\_synset) per word, first sense only, with compress\_count ported verbatim from the Python file so the two sides agree by construction, not by convention.

The cross-check that they actually do agree: wntest.c, run against the real C WordNet library, bank’s first noun sense — HYPERPTR=1, HYPOPTR=2 — exactly matching Python’s {'hypernyms':1, 'hyponyms':2} for bank.n.01 (verified 2026-08-25).

## 6.2 Folding into monad3\_c.bin

VAPMIP/monad\_combine.py is where the already-built WordNet table joins the rest — confirming the sequence: the box-kite/WordNet piece existed first, as its own file, before anything was combined:

```
def read_boxkite_c(path=DEFAULT_WORDNET):          # c_monad_wordnet.bin
    ...                                             # BXKT header + BoxKiteEntry[n]
    return {word: {'pos', 'offset', 'vec19', 'depth_weight'}}

cm = CombinedMonad(english=<monad.bin state>,
                   wordnet=read_boxkite_c(),
                   phonetic=read_phonetic())
write_c(cm, path)    # -> monad3_c.bin
```

write\_c takes the **union** of all three sources’ vocabularies (sorted(set(eng.words) | set(cm.wordnet) | set(cm.phonetic))) and emits the single fixed-offset, mmap-able file every notebook so far has loaded directly — eng: (β/E/A-matrix), wn: (this section’s BoxKiteEntry table), phon: (§6.3).

## 6.3 phon:, in one line

Checked directly: monad\_phonetic.bin is built independently, from nltk.corpus.cmudict (real ARPAbet pronunciation + stress), keyed only by word spelling — no synset, no WordNet relation, anywhere in its construction. No WordNet wiring to report, so

it stays a sibling section in the packed file and out of scope here. One note worth keeping, since it's a real, stated limitation: ARPAbet is ASCII by convention; a move to Unicode IPA is future work, not this file's current form.

## 6.4 Not static

One thing worth saying plainly and no more: `monad3_c.bin` is a **knowledge store**, not a frozen table — the Monad's live learning step keeps deepening it, and a word only reaches "verified" status once it's been seen from both directions, the WordNet entry and real usage. The mechanism itself — what "learn" actually computes — is out of scope for this section by design (§ note above); its role here is just this: **the file this section built keeps growing**, it isn't a one-time export.

## 6.5 The update mechanism itself, in C

`notebooks/06.5_monad3c_update_mechanism.ipynb` — a genuine C-kernel notebook (jupyter-c-kernel, each cell standalone, compiled and run with gcc, not a Python restatement), covering §6.4's "continuing growth" claim at the level Cody asked for: the real update law (`monad_learn_ex`'s  $\beta$ -deepening, `prose_seen` ladder, A-matrix 2D inverse-distance coupling — `PtolC/monad.c`, real constants from `PtolC/ptolemy.h`), and only what the daemon *sends* to trigger it (message class + weight, `external=1.5/internal=0.9`) — not the daemon's own socket/FIFO/spool plumbing. Separate from and complementary to the Python `Crank.learn` shown for CS-paper readability elsewhere.

Two real findings surfaced just by running the extracted logic, both flagged rather than fixed: - **MONAD\_BETA\_SAT = 7.552** (`ptolemy.h`), not the  $(0, 1]$  range `monad_bin/SPEC.md` states for  $\beta$  — a live discrepancy between the spec and the actual C constant. - **The prose\_seen ladder is order-dependent, asymmetrically:** WordNet-then-prose reaches 3 ("verified common"); prose-then-WordNet only reaches 2, silently losing the earlier prose sighting — the `NS_FT_WORDNET` branch overwrites unconditionally rather than checking prior state.

---

**Production note (Cody, 2026-09-20):** `boxkite_bin.h/dump_boxkite_bin.c/wntest.c` are native C, and per Cody: "the Monad and the Monad Harness [are] strictly C these days" — the Python monads (`monad.py`, `rotary_rerun_boxkite_monad.py`) are for testing and CS- paper enumeration, not the live system. Notebook 06 should be a genuine **C-kernel notebook** (Cling/Clang-style, as done before for other notebooks in this project) running this section's actual C, not a Python restatement of it.

---

## 7. Reading `monad3_c.bin`'s WordNet relationships

§6 built the store; this section is only the **reading** mechanism — how a synset's 19 stored relation counts become one address. Nothing here touches the box kite yet; it's the input the pencil/box-kite section consumes directly, not developed further here on purpose.

## 7.1 Nineteen relations, one prime each

VAPMIP/wordnet\_boxkite.py: the 19 relation types this store carries (RELATION\_METHODS) each get their own fixed prime line (CONTEXT\_PRIMES), starting just above the spelling tier's ceiling:

```
RELATION_METHODS = ['hypernyms', 'instance_hypernyms', 'hyponyms',
                    'instance_hyponyms', 'member_holonyms', 'substance_holonyms',
                    'part_holonyms', 'member_meronyms', 'substance_meronyms',
                    'part_meronyms', 'attributes', 'entailments', 'causes', 'also_sees',
                    'verb_groups', 'similar_tos', 'topic_domains', 'region_domains',
                    'usage_domains'] # 19
CONTEXT_PRIMES[:19] = [73, 79, 83, 89, 97, 101, 103, 107, 109, 113,
                      127, 131, 137, 139, 149, 151, 157, 163, 167]
```

## 7.2 Nineteen counts → one integer, exact

$\text{context\_code}(v) = \prod_i \text{CONTEXT\_PRIMES}[i]^{v[i]}$  — unique factorisation makes it recoverable exactly, by construction, checked against the live store rather than trusted on the math alone. Loaded **146,743 words** carrying a stored 19-vector directly from monad3\_c.bin:

loaded 146,743 words carrying a stored 19-vector, in 0.44s

```
example: "'hood"  vector=[1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1]
          context_code=12191  recovered=[1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1]
          match=True
```

'hood's vector has exactly two nonzero slots — hypernyms (position 0) and usage\_domains (position 18) — and  $12191 = 73^1 \times 167^1$ , the product of exactly those two primes. Factoring back recovers the same vector, exactly, on this and (per the paper's own G1 gate) the full store.

## 7.3 One integer → one real — the gamma fold

context\_code is still a large integer, one per distinct relational *shape*, not yet the single continuous scalar the rest of this series carries context as:

```
log_code      =  $\sum_i v[i] \cdot \ln(\text{CONTEXT\_PRIMES}[i])$ 
gamma_radial =  $\tanh(0.5 \cdot \ln(\text{log\_code} / \text{LOG\_ANCHOR}))$ 
```

tanh/atanh round-trip log\_code to machine precision; log\_code itself is injective on real vocabulary by the same unique-factorisation argument as §7.2, one level up. Measured on a 20,000-word sample:

sample: 20,000 words

gamma\_radial round trip (tanh → atanh) max rel err: 8.00e-16

distinct log\_code values seen: 559

19-vector collisions on a repeated log\_code: 0

559 distinct values over 20,000 words is expected, not a defect — many words share an exact relational shape (§7.1's earlier finding, hilbert/ irrawaddy/new\_york among

them); the fold is checked *within* each shape, not claimed to separate words that were never distinct in the first place.

## 7.4 Combined with spelling — one prime per word, both halves recoverable

§5's `spelling_code` uses `LETTER_PRIMES` ( $\leq 71$ ); `context_code` uses `CONTEXT_PRIMES` ( $\geq 73$ ) — disjoint by construction, checked directly:

```
LETTER_PRIMES max: 71    CONTEXT_PRIMES min: 73
disjoint by construction: True
```

so `full_code = spelling_code(word) × context_code(v)`, `full_addr = next_prime(full_code)`, `delta = full_addr - full_code` never has an prime that's ambiguous about which half it came from. Measured:

286/286 exact full round trip (spelling + full 19D context, from ONE prime)

**Flag:** the current README.md §7 states this as “300/300” — the notebook's own recorded run says **286/286**. One of the two is stale; worth reconciling before this section ships (likely just an earlier, smaller sample size than the run recorded here).

One full worked example, end to end:

```
word:                'ubermensch'
context vector:      {'hypernyms': 1}
spelling_code:       2496201486940533516353505421933678773517334546856593090764566773076164
context_code:        73
full_addr:           1822227085466589466938058958011585504667654219205312956258133744345599
delta:               179
```

One prime. Both the exact spelling and the full 19-dimensional WordNet relational signature of the word, recoverable from (`full_addr`, `delta`) alone.

## What this section is not

Not the box kite, not the pencil. `gamma_radial`, exactly as folded here, **is §9's wind-speed** — not an input analogous to it, the same number. §9.1 shows the recovery running the other direction: given only (`full_addr`, `delta`) and the word's own spelling, `gamma_radial` comes back out with no separate WordNet read at all. How that scalar selects a chart or drives a deformation is §9's job, not this one's — this section only has to show the fold is exact and reversible, which it is.

---

## 8. Cost, measured — and why there was no cost function to measure against

### 8.1 The claim, scoped honestly

This is not “backpropagation is wrong.” Backprop remains, right now, the best tool available for a large class of problems — OCR, CNNs generally, anything where a labeled dataset and a differentiable architecture already exist and the compute to train

it is available. The claim here is narrower and practical: **backprop is a hard wall for training a neural network on a laptop**, and that wall is what keeps AI design out of the hands of people doing hobby work at home — not a wall of correctness, a wall of compute. Millions of cost-function evaluations, a stored computational graph, a backward pass through it — that is real, unavoidable work, and it does not fit in an evening on consumer hardware. The engineering goal of this whole project was to make that wall optional, not to declare it wrong.

## 8.2 What a scalar cost function can't see

A trained network's cost function reduces the entire state of the system, at every step, to one number — how far from the target. That scalar has no field structure: it cannot express *which direction* is degenerate versus which direction actually carries information, only *how far* the current guess is from a fixed target. Gradient descent then has to rediscover the local shape of the loss surface step by step, from that one number and its immediate slope, which is exactly why ill-conditioning and saddle points are real, well-documented failure modes of first-order methods (Newton's method, natural gradient, and Hessian-aware saddle-escape methods are the established fixes — all of them use curvature/eigenstructure the plain gradient never looks at). None of that is new here; it's the standard critique, cited as such.

## 8.3 The actual design: intent and desire, forward only

The mechanism this paper documents does not evaluate a cost function at any step. A word's address is computed once, directly (§3–§5); its 19-dimensional WordNet relational signature is read once, directly (§7); nothing is compared against a target and adjusted backward. Two things carry forward instead — **intent** (what the code/maths *function* actually does at this step) and **desire** (the code/maths *jurisdiction* it's allowed to act within) — the same two terms this project's own Lagrangian framing already uses elsewhere, given here their plain engineering translation. Forward propagation, in this sense, is not a metaphor layered on top of the box-kite mechanism — **the scalar box-kite context propagation described in §3–§7 of this paper already is one**, end to end, and is cited here as exactly that: a working instance, not an argument that one could work.

## 8.4 A real precedent with no cost function at all

The design origin for this predates any of this project's own maths: the classic Info-com/Zork-style text-adventure **sentence parser** — a static readout against hardcoded lexicographical data (verb tables, noun tables, fixed grammar rules), zero training, zero cost function, zero gradient of any kind. It answers a question directly by table lookup, every time, deterministically. What this project adds to that model is not a cost function — it's replacing Zork's *flat* hardcoded tables with a *relationally rich* one (the 19D WordNet vector, the box-kite structure it can be read against), while keeping the same zero-training, zero-backward-pass shape the parser always had.

## 8.5 The anchor, kept minimal on purpose

The only zero-divisor structure this section needs is the anchor: a fixed point on the Real numberline ( $e_0$ , already established as the box kite's anchor in §3.2) that the box-kite's own zero-divisor structure turns around, hyperdimensionally, while the anchor itself stays fixed. Nothing about *why* that structure is stable, or what its eigenvalue decomposition looks like at a zero divisor, is needed here — that's real, and it's VAPMIP/Ainulindalë territory (the Mind's Eye's own "Zero Divisor Reframe," where a thought-pathway only gets promoted to long-term memory once it's checked both mathematically correct *and* contextually sane — a real, separate design principle, not developed in this paper). What's worth saying plainly: any 2D/3D rendering of that structure on a screen is a flattened shadow of something hyperdimensional and never actually static — the same caution this project already keeps on record as Flattening Syndrome.

## 8.6 Cost, measured

The comparison that *is* this paper's to make — what the already-shipped mechanism (§3–§7) actually costs, measured, on real hardware:

|               |                                                                                                       |
|---------------|-------------------------------------------------------------------------------------------------------|
| read path:    | ~600 flop/token (one sedenion-scale product + one sparse A-matrix row)                                |
| ingest fold:  | $1.8 \times 10^5$ words/s, $\approx 38 \mu\text{J}/\text{word}$ @ 7W (reference machine, single core) |
| native floor: | $\approx 115$ ns, $< 1 \mu\text{J}/\text{word}$                                                       |

against the standard forward-FLOP identity for a dense transformer ( $2 \cdot N_{\text{params}}$  multiply-accumulates/token,  $\sim 1$  J/token at 70B params,  $10^{-11}$  J/flop datacentre-effective) —  **$10^4$ - $10^6 \times$  cheaper per query**, and the gap is structural, not an optimisation: the addressed structure regenerates its answer from a fixed ruler; the materialised field is re-swept in full every time because the answer lives in the weights.

**Reference machine, resolved:** two real machines, not a conflict. The **HP EliteBook 820 G3** (i7-6600U) was the original — it “died a hard and painful death” (screen delaminated, electron gun failed) and was superseded, not deprecated, by the **Lenovo ThinkPad X1 Carbon 6th gen** (i7-8550U, VAPMIP/docs/SYSTEM\_SPECS\_ThinkPad\_X1\_Carbon\_6th.md, snapshot 2026-07-31) — confirmed genuinely the X1 Carbon via DMI, even though the machine's own hostname string still reads `rendier-HP-EliteBook-820-G3`, carried over from the migration. The §8.6 numbers above are the X1 Carbon's, correctly attributed in `engine/energy_bench.py`'s own header; earlier project benchmarks (the 25,000-zero golden-ratio work) are the EliteBook's, a separate, earlier machine, not these numbers. No correction needed to the measured figures — only worth stating both machines' provenance explicitly if this section names hardware age at all, since the EliteBook is the one that's actually old.

**Back Propagation = Bad = Hard = Inefficient = Work = Hard Boundary.**  
**Forward Propagation = Good = Easy = Free = Less Work** — Cody's own framing, kept as a direct quote. The “15 year old laptop” line alongside it is a deliberate

retorical placeholder, not a literal hardware-age claim (neither the X1 Carbon nor the EliteBook is actually 15 years old) — the point it’s making is real regardless: whatever the exact machine or its exact age, it’s a dinosaur next to a current top-of-line laptop, and it still runs this fine. The measured numbers above are what actually back the claim; the “15” is the easy-to-grasp version of it, not a spec.

---

## 9. The Pencil — windspeed, and what it takes to run the box kite today

Scoped deliberately narrow, per the paper’s own rule: only what’s necessary to run the box-kite mechanism as the current monad actually uses it. The fuller exploration this pencil work opened up — recorded in full in VAPMIP/Boxkite-Catalog.txt — is real, substantial, and explicitly **not** developed here; it closes the paper as “in development,” §11.

### 9.1 The windspeed, finalized

Two real candidate windspeeds were tested side by side this pass (weighted- operator probe, all 7 struts) before this section could be written honestly. **gamma\_radial, recovered from a word’s own stored address, is the windspeed for this paper** — exact, deterministic, needs nothing but the word’s text and its (full\_addr, delta):

```
full_code = full_addr - delta           # exact, §7
spelling  = spelling_code(word)         # from the text alone
context_code_recovered = full_code // spelling # exact integer division –
                                           # disjoint prime tiers, §7.
windspeed = gamma_radial(context_code_recovered) # the real-valued fold, §7.3
```

Verified live against a real word: windspeed("tree") = -0.151155.

The other real candidate — A-matrix basin drift — is **not** this paper’s windspeed. It needs a live, mutating, corpus-dependent store, which is exactly wrong for a claim about forward propagation from an address alone. It has a real, separate home: renamed “the Newton basin locus contextual flow,” it’s the windspeed for sentence *construction* — a different, later engineering pass (Mind’s Eye), out of scope here. One name, two genuinely different jobs; kept apart on purpose.

### 9.2 The pencil, attached

pencil(s) — 7 factorisations of one relation, ValaQuenta/modules/ box\_kite/maths.py, built and verified this project. For strut 1:

```
>>> bk.pencil(1)
[(2,3), (4,5), (6,7), (8,9), (10,11), (12,13), (14,15)]
```

Each station is a pair of PG(3,2) points XORing to the anchor. This is the minimal slice needed here — just enough that a windspeed has something to act on. The fuller pathway/portal structure over the pencil is Catalog material, §11.

### 9.3 What the deformation actually does — corrected twice, now honest

Two real mistakes, corrected in the record rather than quietly fixed, because both changed the answer:

**Wrong coordinate frame.** An initial check of pencil\_hyperstring.md’s “one flex mode” claim used ordinary Cartesian 3D rigidity theory (Maxwell counting:  $\dim(\text{flex}) = 3 \cdot 6 - 6 - 3 = 9$ , exact, matching an earlier, independent notebook finding). That arithmetic is correct and irrelevant — this project’s own standing rule is that sedenion/box-kite maths is native to **radial complex spherical polar coordinates**, never Cartesian internally. Confirmed once corrected: pairing an Assessor’s  $(d_+, d_-)$  as one native complex number (the same move already verified elsewhere in this project’s  $\sigma_{\text{RB}}$  work) is the right representation, not two independent Cartesian reals.

**Wrong aperture.** A second check used bare Cayley-Dickson basis vectors  $(e_2, e_4, \dots)$  as pencil-station generators — exactly as pencil\_hyperstring.md literally specifies (“a unit sedenion”). Checked directly: every one of the 256 possible basis-vector products is *exactly*  $\pm 1$  times another basis vector (zero exceptions) — a closed, quantized lattice.  $H = \text{Re}(\Pi)$  landing on  $\{-1, 0, 1\}$  under this representation, found originally in notebook 04, was therefore never evidence that the box kite’s own structure is discrete. It’s what multiplying quantized lattice points together always does. Opening the aperture — the same generator varying continuously along its own great circle instead of a discrete pick — gives smooth, exact variation:  $\text{Re}(\Pi) = -\cos(\phi)$ , verified.

Both corrections came from the same discipline, now written up as its own skill (observer-position) precisely so this doesn’t get re-discovered the hard way a third time: track *which coordinate frame* (orientation) and *how much of the structure*, at *what resolution* (aperture) a result was taken from before trusting it.

### 9.4 H, conserved — a real, partial result

Once the aperture is opened,  $H$  is not merely continuous — it is **exactly conserved** along one specific, non-arbitrary path: each pencil station’s phase advancing at a rate proportional to its own position in the pencil ( $\phi_k \propto (k+1)/7$ ), moving through all seven stations *in pencil order*. Checked against three other joint-motion rules (uniform rate, reversed order, random per-station rates) — none of them conserve  $H$ ; only the pencil’s own order does. Not a trivial fact about any coordinated motion — a specific one.

Checked across all 7 struts, and here the honest result is a real, partial one, not a closed law: **H conserves exactly on struts 1, 3, and 6, and drifts on struts 2, 4, 5, and 7** — confirmed against two different generator conventions (which endpoint of each station pair is taken “first”), so it’s a fact about the strut, not an artifact of that choice. *Why* those three specifically is open — checked and ruled out that they form a closed Fano-plane line ( $1 \oplus 3 = 2$ , not 6) — kept honest as an unresolved structural question for the Catalog, §11, not forced to a premature answer here.

## 9.5 The crossing, precisely — J\_2, not J\_N

One more correction worth keeping, because it's now exact rather than descriptive. The Joukowski map used at a strut crossing,  $\zeta \mapsto \zeta + c \cdot \zeta^{-1}$  ( $c = H/q(w)$ ), is **not itself an involution** —  $f(f(\zeta)) \neq \zeta$ , checked directly. It is the 2-to-1 projection of a genuine order-2 symmetry underneath it,  $\zeta \leftrightarrow c/\zeta$  (not simply  $\zeta \leftrightarrow 1/\zeta$  unless  $c=1$  — verified the general form exactly:  $f(\zeta)=f(c/\zeta)$ , and  $c/(c/\zeta)=\zeta$ , both exact). That symmetry is J\_2, the same order-2 fold already established elsewhere in this project (the Red/Blue swap, the (I|0) two-stroke engine) — **not** J\_N, the separate order-4 map this project also has on record. The crossing itself is the fixed point of that J\_2 fold,  $\zeta = \sqrt{c}$  — which is exactly why a fixed windspeed can pin a crossing at all: an order-2 involution has one clean fixed point; a bare continuous deformation with no such symmetry would not.

## 9.6 What runs today, in the current monad

Honest status, not aspirational: the windspeed recovery (§9.1) is exact arithmetic, verified in Python, portable to C directly — not yet ported. What genuinely runs in `ptol.c` today is narrower: `monad3_lookup()` `mmaps` `monad3_c.bin` and reads a word's real 19-dimensional relation vector straight out of the packed store by raw pointer arithmetic (-M, byte-exact against `boxkite_bin.h`, §6) — the box-kite object is real and live in the C binary, but the windspeed-driven deformation this section derives is Python-verified THEORETICAL:CALCULATED, not yet the thing `ptol.c` runs. Stated plainly rather than implied otherwise: this section's centerpiece is the *mechanism*, checked and ready to port; the porting itself is the next, separate step.

### What this section is not

Not the full sail-pressure-to-strut-angle mechanics Cody's own model describes (unbuilt — the spectral checks above are a proxy, not that geometry). Not the fuller pathway/portal exploration, the Blackjack subgroup, or the {1,3,6} question's resolution — all Catalog material, named and pointed at, §11, not developed here.

---

## 10. Conclusion — what the scalar actually does, proven and open

### The claim this paper set out to prove

**Proven, exactly, on live data:** a word's full 19-dimensional WordNet relational signature is recoverable from one prime address and its literal spelling, with no separate WordNet lookup. `context_vector`→`context_code` factors back exact on the full 146,743-word store (§7.2). The combined (spelling, context) address round-trips exact, 286/286, on a live sample (§7.4). And read the other direction — the one this paper is actually about — **windspeed (gamma\_radial) recovers out of (full\_addr, delta) and the word's own text alone**, dividing out the spelling component computed fresh from the letters, with no context stored anywhere except in that one scalar (§9.1). That is the proof of concept: the single scalar number *does* recreate the exact structure that leads to the 19D context relationships, starting from nothing but the literal spelling

of the word. Nothing about this claim is aspirational — it is measured, exact, and reproducible from the notebooks cited throughout.

### What is genuinely working

- The address itself (§3–§5): deterministic, four months unmodified, measured live at 347,119 words.
- The domain it's built on (§4): 313 as the sieve's own extinction boundary — not a round number, a fact about  $p^2 \leq N$  — grounding “the first 65 primes are letters.”
- The spelling fix (§5): order-sensitive, exact for the  $\leq 20$ -letter words the tier is built for, and the failure past that is the *predicted* arithmetic wraparound, not a mystery.
- monad3\_c.bin (§6): the real WordNet box-kite table, built and cross-checked C-vs-Python exact, and confirmed live in ptol.c today via -M's byte-exact struct read — not a paper design, a running binary.
- The context read and its scalar fold (§7): exact, both directions.
- The windspeed recovery (§9.1): exact, and — corrected twice in the process, both corrections kept in the record — genuinely continuous once represented in this project's own native coordinates rather than a Cartesian or basis-vector-quantized stand-in.
- A real, non-trivial conservation law (§9.4):  $H$  exactly conserved along one specific, non-arbitrary path through the pencil, on 3 of 7 struts.
- The cost claim (§8): measured,  $10^4$ – $10^6 \times$  cheaper per query, structural not optimisation.

### What needs work — named honestly, not hidden

- **$\Phi_w$ , the full deformation law**, is still open. What §9 adds is real progress — the object is continuous, not discrete, and conservation holds on a real subset of struts under a specific ordering — but the complete map from  $(H, w)$  to all six Assessor coordinates, for every strut, is not built.
- **Why struts {1,3,6} conserve and {2,4,5,7} don't** is unresolved. Checked and ruled out the obvious candidate (a Fano-plane line); the real answer likely lives in the pencil's own  $(a,b)$  pairing structure, not the index set, and needs a properly-targeted test this pass didn't finish.
- **Cody's own sail-pressure-to-strut-angle mechanics** — deformable sails fixing two struts' relative position at the crossing — remains unbuilt. Everything in §9.3–§9.4 is a spectral/algebraic proxy for that geometry, not the geometry itself.
- **The C port.** The windspeed recovery is exact arithmetic, verified in Python, and has no dependency that would block porting it into ptol.c directly — it just hasn't been done yet. Today's live C touchpoint stays at -M's read of the stored vector, not a computed reconstruction.
- **The Scale engine's contextual flow**, and the sentence-construction windspeed (“the Newton basin locus contextual flow”) this paper deliberately excluded, are both real, both wanted, and both separate, later engineering — not gaps in this paper's own claim, but named here so the boundary is visible. See §11.

## The honest shape of it

This paper proves the *address* half of its own claim completely: context propagates forward, exactly, from one scalar and a spelling, with nothing stored in between. It proves a *real, structured* opening piece of the *geometry* half — continuous where earlier passes found (wrongly) discrete, conserved on a genuine subset of the structure — without proving the whole geometry. That is a true, checkable, and useful place to stop a first paper.

---

## 11. Direction of engineering research — in development

Everything in this section is named, some of it measured, none of it claimed as part of this paper's proof. It exists so the next work has a place to start from, and so a reader who finds this paper first can see where the rest of it is heading before any of it is finished.

### The Boxkite Catalog

VAPMIP/Boxkite-Catalog.txt — a full session's inventory of box-kite structure, most of it real and verified, none of it developed here: per-Assessor and per-strut invariants; torsion, circulation, and lift (Kutta-Joukowski, split 53 positive / 31 negative / 21 zero across all 105 four-cycles); portal/transition dynamics (confirmed discrete, not continuous, two independent ways); the connectivity of all seven charts at exactly  $e_0/e_8$ ; and the group that ties it together —

**The Blackjack subgroup.** The 21-element subgroup of  $PSL(2,7)$  that preserves zero-divisor structure while staying transitive across all seven struts (2026-08-13\_apex\_path/psl27\_strut\_action.py, verified 2026-09-17). Named here for the first time as such; the atlas of seven charts is connected by this group action, not by any edge.

### The Pencil's own open questions

Why  $H$  conserves on struts  $\{1, 3, 6\}$  and not  $\{2, 4, 5, 7\}$  (§9.4/§10) — the most immediate open thread from this paper's own work, likely resolved in the pencil's  $(a, b)$  pairing structure rather than the strut index. And the deformation law itself,  $\Phi_w$  — Cody's own model: deformable sails connecting to the ends of the struts deform in a constant way, placing pairs of struts in a fixed relationship to one another at the crossing. Not yet built; §9.3–§9.4 measure a spectral proxy for it, not the mechanics.

### The sentence constructor

rotary\_rerun\_boxkite\_monad.py — a real, live BoxKite object (`ValaQuenta.modules.box_kite`, genuinely 7 struts / 42 Assessors, not a metaphor), built once via `BoxKite.between(eye, hands)` as “*the relational language spoken by both MindsEye and PapersHands*,” signed so a third party can verify which two subsystems co-authored it. The real sentence creator (VerbNet sails + WordNet hypernym-closure fill + a SELRESTR gate,

engine/grammar/) is live as of 2026-09-11, confirmed running end to end this session (voice:creator in a real Chat-tab transcript, a genuine constructed sentence, not one of the eleven canned templates it replaced). None of this paper's windspeed feeds it yet.

### **The Newton basin locus contextual flow**

The other windspeed tested in this paper's own §9.1 — A-matrix basin drift — was deliberately not used here. It is real, corpus-dependent, and usage-sensitive in exactly the way this paper's corpus-free windspeed isn't, which is precisely why it's the right scalar for word *selection* inside live sentence construction rather than for address-only context propagation. Renamed for this purpose, it becomes the constructor windspeed for a later, separate engineering pass on the Mind's Eye.

### **Mind's Eye as a box kite**

A stated direction, not yet built: making the Mind's Eye itself a box kite, so that rehearse (raising salience on a candidate) is tracing rings — closed walks — on the kite's own structure, archive is committing whichever ring won, and audience-appropriate word choice is a genuine geometric constraint (steering the walk within a listener's own reachable vertices) rather than a filter bolted on afterward. Sentences as **rings inside one box kite**, not rings of box kites — a real correction to an earlier framing, kept as a parallel, not a replacement, to the older per-word box-kite-as-database model this project also still uses (§6). Connects to the Zero Divisor Reframe already named in the Mind's Eye's own design: a thought-pathway is promoted to long-term memory only once it is both mathematically correct *and* contextually sane — two separate gates, kept separate on purpose.

### **A methodology note, for whoever picks this up next**

Two real mistakes in this paper's own working sessions — analysing a Native-Space object in Cartesian coordinates, and quantizing a continuous generator onto a 16-point lattice without noticing — cost real time before being caught. Both are now a standing skill (~/.claude/skills/observer-position/), built specifically so hyperdimensional work states its coordinate frame and its aperture before trusting a result, the same discipline a camera pose or a viewport offset already gets in lower-dimensional work. Offered here as process, not results, because the mistakes were real and the fix generalises past this one paper.

---

## **12. Provenance and attribution**

Everything in this paper carries one of five labels, applied consistently rather than asserted once and forgotten. Consolidated here — previously scattered across notation, disambiguation, and a per-component table — because attribution is ongoing, not a one-time note, and belongs in one place a reader can check against every section.

## The labels

- **ESTABLISHED** — mathematics or computer science that predates this project, cited to its source.
- **OURS** — code or a specific design choice built for this project, not published elsewhere.
- **FIRST STATED HERE** — a specific claim or construction, dated, not found stated this way anywhere else checked.
- **THEORETICAL** — designed, or partially attempted, not yet reduced to running, verified code.
- **THEORETICAL:CALCULATED** — a **THEORETICAL** component for which code has nonetheless computed a concrete, reportable result — a measurement, a boundary condition, a failed literal construction — without the component itself shipping.

## How the code came first

Every mechanism in this paper — the address, the spelling fix, the 19D context fold, the windspeed recovery — was built directly against the problem, then checked against the literature afterward, not designed from it. Correspondences to established mathematics (projective-plane combinatorics, Gödel positional encoding, Miller-Rabin primality, sedenion zero-divisor structure, the Joukowski transform) were noticed once the code already worked. Where a mathematical name appears in this paper, it is a label applied after the fact for a reader who already knows it — it played no role in how the code was written.

## Provenance by component

| component                                     | provenance                                                                         | status                      |
|-----------------------------------------------|------------------------------------------------------------------------------------|-----------------------------|
| token → address (Horner, next_prime, π)       | <b>OURS</b> — monad.py, unaltered since 2026-05-27                                 | ships                       |
| the 313 sieve boundary                        | <b>ESTABLISHED</b> — classical sieve theory; “first 65 primes are letters” reading | <b>OURS</b>                 |
| spelling → prime, order-preserving            | <b>OURS</b> , marked provisional in-source                                         | ships                       |
| WordNet box-kite table (monad3_c.bin)         | <b>ESTABLISHED</b> WordNet; storage/build <b>OURS</b>                              | ships, C-verified           |
| 19D context fold, context_code/gamma_radial   | <b>ESTABLISHED</b> WordNet relation vocabulary; encoding <b>OURS</b>               | ships, exact round trip     |
| windspeed recovery (full_addr,delta,spelling) | <b>OURS</b> , <b>FIRST STATED HERE</b> 2026-09-20                                  | verified, not yet in ptol.c |

| component                                                      | provenance                                                                                                                          | status                                       |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| the box kite — 42 Assessors, 7 octahedra, PSL(2,7)             | <b>ESTABLISHED</b> — de Marrais (2000)                                                                                              | cited, not re-derived                        |
| the pencil — 7 factorisations of one relation                  | <b>ESTABLISHED</b> — projective geometry (PG(3,2)); edge framing <b>OURS</b>                                                        | ships                                        |
| the Blackjack subgroup (21-element, transitive, ZD-preserving) | <b>FIRST STATED HERE</b> 2026-09-17                                                                                                 | verified                                     |
| $\Phi_w$ , the Joukowski deformation law                       | <b>ESTABLISHED</b> — Joukowski (1910), elastica (Euler, 1744), Kutta-Joukowski; tether/wind-inflation <b>FIRST STATED HERE</b>      | THEORETICAL, partially :CALCULATED this pass |
| the J <sub>2</sub> /J <sub>N</sub> crossing identity           | <b>ESTABLISHED</b> (classical complex analysis) — the identification of <i>which</i> involution <b>FIRST STATED HERE</b> 2026-09-20 | verified exactly                             |
| H conservation on struts {1,3,6}                               | <b>FIRST STATED HERE</b> 2026-09-20                                                                                                 | measured, unexplained                        |
| the cost comparison                                            | <b>OURS</b> measurement; the dense-transformer FLOP identity <b>ESTABLISHED</b>                                                     | measured                                     |
| observer-position methodology                                  | <b>OURS</b> , 'FIRST STATED HERE** 2026-09-20                                                                                       | in use                                       |

## Licensing

The box-kite context-hashing method and all code described in this paper — the addressing pipeline, spelling\_code, context\_code, the gamma fold, the windspeed recovery, and every notebook this paper cites — is released under the **GNU General Public License, version 3 (GPLv3)**. Free to use, study, modify, and redistribute, for research and commercial purposes alike, under that license's terms.

This paper describes an address, not the whole system it is a component of. Nothing in the broader, undisclosed system that address feeds is required to use, verify, or extend anything described here — every claim in this paper is complete and reproducible from what is cited in §13, independent of it.

## References

1. de Marrais, R. P. C. (2000). *The 42 Assessors and the Box-Kites They Fly: Diagonal Axis-Pair Systems of Zero-Divisors in the Sedenions' 16 Dimensions*.

arXiv:math/0011260.

2. Moreno, G. (1997/98). *The zero divisors of the Cayley-Dickson algebras over the real numbers*.
3. Joukowski, N. (1910). The Joukowski transform / airfoil mapping.
4. Euler, L. (1744). The elastica — equilibrium of a loaded flexible rod.
5. Kutta, W. M.; Joukowski, N. The Kutta-Joukowski theorem.
6. Smith, P. H. (1939). The Smith chart.
7. Patterson, D.; et al. (2021). *Carbon Emissions and Large Neural Network Training*.
8. Maxwell, J. C. (1864); Laman, G. (1970). Combinatorial rigidity counting — cited as the analogue that turned out to be the wrong frame for §9.3, kept in the record for exactly that reason.

**Cody Michael Allison** (Michael Rendier). Correspondence: the.wandering.god@gmail.com · GitHub: github.com/michaelrendier · ORCID: 0009-0007-7239-6760.

## 13. Code appendix

The executable form of every result in this paper. Not the `repo_appendix` generally — this section is the curated path through it: what to run, in what order, to reproduce §3 through §11 from nothing but this repository and its two sibling repos (VAPMIP, ValaQuenta).

### 13.1 The notebooks

| notebook                                            | proves                                                                                                                | section |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|---------|
| notebooks/01_semantic_prime_hashtables.ipynb        | the Hashtables (Horner $\rightarrow$ next_prime $\rightarrow$ $\pi$ ), the 6543-vs-6542 anomaly, the U+200B collision | §3      |
| notebooks/02_the_313_sieve_313s4999sieve.ipynb      | the 313s4999sieve, extinction boundary, derived twice independently                                                   | §4      |
| notebooks/03_phonetic_prime_hashtables.ipynb        | the Hashtables' anagram collision, the Gödel-positional fix                                                           | §5      |
| notebooks/05_wordnet_19d_contextual_addresses.ipynb | the contextual_addresses, exact both directions, the combined address                                                 | §7      |
| notebooks/06.5_monad3c_update_monad_learn_ex.ipynb  | the monad_learn_ex — C-kernel, every cell actually compiled and run                                                   | §6      |

| notebook                                                           | proves                                                                                                  | section |
|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|---------|
| repo_appendix/wind_speed_recovery/ (11 scripts + README, numbered) | the wind speed recovery, the coordinate-frame and aperture corrections, the {1,3,6} conservation result | §9      |

notebook 04 (boxkite\_pencil\_hyperstring\_wind\_speed.ipynb) is cited throughout §9 as the origin of the discreteness anomaly this paper’s own work later resolved — read alongside 07\_unflattened\_continuous\_H.py above for the corrected version of the same question.

### 13.2 The build — WordNet into the box kite, in C

PtolC/boxkite\_bin.h — the packed record, byte-identical on both sides of the build:

```
typedef struct {
    char    word[32];
    uint8_t pos;
    uint32_t synset_offset;
    int16_t vector[19];
    float    depth_weight;
} BoxKiteEntry;
```

PtolC/dump\_boxkite\_bin.c — built from the real WordNet C library, not a reimplementation:

```
static void synset_context_vector(SynsetPtr syn, int16_t out[N_RELATIONS]) {
    int raw[MAXPTR + 1];
    memset(raw, 0, sizeof(raw));
    for (int i = 0; i < syn->ptrcount; i++) {
        if (syn->pfrm[i] != 0 || syn->pto[i] != 0) continue;
        int t = syn->ptrtyp[i];
        if (t >= 0 && t <= MAXPTR) raw[t]++;
    }
    for (int r = 0; r < N_RELATIONS; r++)
        out[r] = compress_count(raw[REL_PTRTYPE[r]]);
}
```

Cross-checked against the Python side (wordnet\_boxkite.py) synset by synset, not assumed to agree — PtolC/wntest.c, verified 2026-08-25.

### 13.3 The read — a word’s address, in the current monad

PtolC/ptol.c, the live touchpoint (-M, byte-exact against §13.2’s struct, no shell-out):

```
const unsigned char *e = g_m3 + g_m3h->off_wn + (size_t)ix[1] * 82u;
uint8_t pos = e[32];
```

```
const int16_t *vec = (const int16_t *) (e + 40);
```

### 13.4 The windspeed recovery, exact

VAPMIP/wordnet\_boxkite.py + repo\_appendix/windspeed\_reconstruction/01\_windspeed\_recovery

```
full_code = full_addr - delta
recovered_spelling = spelling_code(word)                # from text alone
recovered_context = full_code // recovered_spelling    # exact, disjoint tiers
log_code = sum(v[i] * math.log(CONTEXT_PRIMES[i]) for i in range(19))
gamma_radial = math.tanh(0.5 * math.log(log_code / LOG_ANCHOR))
```

### 13.5 The box kite itself

ValaQuenta/modules/box\_kite/maths.py — pencil(s), box\_kites(), assessors(), chart\_of(); rotary\_rerun\_monad.py::BoxKite.between(eye, hands) — the live, signed, jointly-authored instance the current monad actually binds to.

### 13.6 Reproduce

```
python3 -m venv .venv && source .venv/bin/activate
pip3 install -r requirements.txt
python3 -c "import nltk; nltk.download('wordnet')"
```

# sibling repos VAPMIP and ValaQuenta checked out alongside this one —  
# every notebook's first cell hardcodes ~/Projects/ThePlace/<repo>; a  
# real portability wrinkle, named in \$1, not yet fixed.

```
jupyter notebook notebooks/
```

repo\_appendix/windspeed\_reconstruction/ scripts are plain python3, ValaQuenta/.venv required for the WordNet-dependent ones (nltk/sklearn ABI conflict on bare system Python — use the venv, not a workaround).